

MVC with Servlets & JSP — Product Registration Assignment

Topics: JavaBeans, Servlet-as-Controller, JSP-as-View, HttpSession, List<Bean>

Q1. Product.java — JavaBean (Model)

```
public class Product {

    private int id;
    private String name;
    private double price;
    private String category;

    public Product() { }

    public Product(int id, String name, double price, String category) {
        this.id = id;
        this.name = name;
        this.price = price;
        this.category = category;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }

    public String getCategory() { return category; }
    public void setCategory(String category) { this.category = category; }
}
```

Q2. AddProductServlet — Builds the Bean, Stores it in Session

addProduct.html (attached) submits name/price/category via POST.

```
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    String name = request.getParameter("name");
    double price = Double.parseDouble(request.getParameter("price"));
    String category = request.getParameter("category");

    Product product = new Product(1, name, price, category);

    HttpSession session = request.getSession();
    session.setAttribute("product", product);

    RequestDispatcher dispatcher = request.getRequestDispatcher("showProduct.jsp");
    dispatcher.forward(request, response);
}
```

(This is the base single-product version; the attached AddProductServlet.java file already contains the Q5 refactored, list-based version described below.)

Q3. showProduct.jsp — Displaying the Bean in a Table (base version)

```
<%
    Product product = (Product) session.getAttribute("product");
%>
<table border="1">
    <tr><th>ID</th><th>Name</th><th>Price</th><th>Category</th></tr>
    <tr>
        <td><%= product.getId() %></td>
        <td><%= product.getName() %></td>
        <td><%= product.getPrice() %></td>
        <td><%= product.getCategory() %></td>
    </tr>
</table>
```

Q4. MVC Flow for Flipkart-style Product Registration

This assignment implements the classic Servlet/JSP MVC pattern:

Role	Component	Responsibility
Model	Product.java	Plain JavaBean holding product data (id, name, price, category).
View	addProduct.html / showProduct.jsp	Collects input from the user and displays the result — no business logic.
Controller	AddProductServlet	Receives the form submission, creates/updates the Model, stores it, and d

Flow:

1. User fills the addProduct.html form (View) and submits it. → 2. AddProductServlet (Controller) receives the POST request, reads the parameters, and builds a Product object (Model). → 3. The Controller stores the Product in the session via session.setAttribute(). → 4. The Controller forwards the request to showProduct.jsp (View) using RequestDispatcher.forward(), which reads the Product from the session and renders it as a table — without any of the JSP page containing business logic itself.

Q5. Refactor — Multiple Products via List<Product> in Session

Instead of overwriting a single 'product' session attribute on every submission, the servlet now keeps growing a List<Product> stored in the session, so every product registered during the session is retained and shown together.

```
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    String name = request.getParameter("name");
    double price = Double.parseDouble(request.getParameter("price"));
    String category = request.getParameter("category");

    int newId = (int) (System.currentTimeMillis() % 1000000);
    Product product = new Product(newId, name, price, category);

    HttpSession session = request.getSession();

    @SuppressWarnings("unchecked")
    List<Product> productList = (List<Product>) session.getAttribute("productList");

    if (productList == null) {
        productList = new ArrayList<>();
    }
    productList.add(product);
    session.setAttribute("productList", productList);

    RequestDispatcher dispatcher = request.getRequestDispatcher("showProduct.jsp");
    dispatcher.forward(request, response);
}
```

Updated showProduct.jsp — loops over the whole list:

```
<%
    List<Product> productList = (List<Product>) session.getAttribute("productList");
%>
<table border="1">
  <tr><th>ID</th><th>Name</th><th>Price</th><th>Category</th></tr>
  <%
    for (Product p : productList) {
  %>
    <tr>
      <td><%= p.getId() %></td>
      <td><%= p.getName() %></td>
      <td><%= p.getPrice() %></td>
      <td><%= p.getCategory() %></td>
    </tr>
  <%
    }
  %>
</table>
```

Result: registering three different products in one browser session (without closing it) now shows all three rows on showProduct.jsp, instead of only the most recently added one.